

DESIGN

Reactive Recursive Submodule Auto-Sync – Risk Research + Design + Phased Plan

Revision: 1 **Last modified:** 2026-07-14T00:00:00Z **Status:** DESIGN DOCUMENT ONLY — research + design + phased plan. **NONE of this is implemented.** No engine code exists for this mechanism; every behavioural component below is PLANNED. This document does not claim any working behaviour (§11.4.6 no-guessing). **Authority:** Operator design directive (2026-07-14) — extend the Docs Chain engine (§11.4.106) with a rock-solid, risk-free, recursive reactive submodule auto-sync capability. **Classification (§11.4.17):** universal (the mechanism references no project-specific hardware; it operates on any git-submodule tree). The ATMOSphere-specific submodule roster, `/mnt/trackN` layout, and owned-org set are consumer-side data per §11.4.35. **Design provenance:** Docs Chain engine docs (`docs_chain/docs/ARCHITECTURE.md` , `CONFIG_SCHEMA.md` , `CONSTITUTION_INTEGRATION.md`); constitution anchors §11.4.113, §9.2, §11.4.84, §11.4.180, §11.4.26, §11.4.188, §11.4.86, §11.4.69, §11.4.107, §2.1, §11.4.28, §11.4.31, §11.4.176–§11.4.182; authoritative git-scm.com sources (see Sources footer).

. Executive summary

The project uses git submodules deeply: the main repo `.gitmodules` declares ~30 submodules (10 owned players + HelixQA dependency chain + hardware libs); the constitution submodule carries its own nested `.gitmodules` (`token_optimizer` + `session_orchestrator`); and the multi-track layout (§11.4.178/§11.4.179) means the SAME submodule can be checked out in many

places across `/mnt/track1..4`. Keeping this graph consistent by hand is error-prone: the #1 git submodule footgun is the **detached-HEAD default** — `git submodule update` leaves a submodule with no working branch, and *"even if you commit changes to the submodule, those changes will quite possibly be lost the next time you run `git submodule update`"* (git-scm.com/book, Source [S1]).

The operator's three rules are:

- **(a)** a submodule tracking `main / master` → regular fetch+pull, then FULLY-RECURSIVE commit+push to ALL upstreams (§2.1), with a clean `git status` invariant on every submodule;
- **(b)** a submodule an operator has DELIBERATELY checked out to a non-main branch → **PIN** that HEAD; NEVER auto-update it;
- **(c)** a change committed+pushed on ANY submodule anywhere → auto-update ALL OTHER copies of that same submodule out-of-box, then commit+push the parent-pointer(s) recursively to all upstreams.

The **core design decision** is to model this as a new set of **node kinds + a git-adapter transform family inside the Docs Chain DAG** (§11.4.106), NOT as a new standalone script. Docs Chain already gives us the exact primitives this needs: content-hash change detection (not mtime, §11.4.86), a Kahn-topological DAG with cycle rejection, declared-authority bidirectional `sync` edges with a "both-dirty ⇒ conflict, no writes" rule (§11.4.6), atomic staging + rename + rollback (§8 / §9.2), `verify` as a read-only deterministic gate (§11.4.50), and per-run captured evidence (§11.4.69). We add: a **git_submodule node kind** whose "content hash" is the recorded gitlink SHA + branch-tracking state; a **per-submodule state machine** (`branch-tracking` vs `pinned` vs `dirty` vs `conflict`); a **fan-out propagation index keyed by submodule remote URL** so all duplicate copies converge; a strict **child-before-parent-pointer ordering** guarantee (mirroring `git push --recurse-submodules=on-demand`, Source [S1]); a **re-entrancy guard** (a run-scoped `.docs_chain/SYNC_IN_PROGRESS` flag + git-lock reaping per §11.4.180) so a sync that commits cannot re-trigger itself into an infinite loop; and an **anti-bluff surface** where every sync run captures a `git_sync_evidence.json` and ships a golden-good/golden-bad self-validated analyzer (§11.4.107(10)).

Top risks (full list in Part B1): detached-HEAD commit loss (R1); auto-updating a deliberately-PINNED branch and destroying operator work (R2 — the rule (b) violation); force-push data loss (R3 — STRICTLY FORBIDDEN §11.4.113); fan-out

inconsistency across duplicate copies (R7); concurrent-track races on one shared object store (R8/R9); child-parent pointer ordering inversion (R11 — a dangling gitlink); hook re-entrancy infinite loop (R12); `.gitmodules` graph cycles / unbounded recursion (R13); and stale `index.lock` / `HEAD.lock` freezing every worktree at once (R16).

Plan: 8 ordered phases (Phase 0 spec → Phase 7 ATMOSphere wiring), each with its scope, touched files, pre-build gate + paired §1.1 mutation + on-host test, and acceptance criteria. Everything below is a PLAN; nothing is built this run.

1

PART B — In-depth risk research

Each risk states the **failure mode** (what breaks for the operator), the **git-level cause** (the precise mechanism), and the **mitigation** the design (Part B2) must implement. Sources are cited as [S1] git-scm Book — Submodules, [S2] git-scm — git-worktree(1), [S3] git-scm — git-submodule(1); full URLs in the Sources footer. Facts not verified against a fetched source this session are marked **UNCONFIRMED** per §11.4.6 (never guessed).

R — Detached-HEAD commit loss

- **Failure mode:** the auto-sync commits inside a submodule that is in detached-HEAD state; a later `git submodule update` re-checks-out the superproject-recorded SHA and the just-made commit becomes unreachable (only recoverable via `reflog`, garbage-collected after `gc.reflogExpireUnreachable`).
- **Git-level cause:** [S1] — *"Git would ... leave the sub-repository in what's called a 'detached HEAD' state ... even if you commit changes to the submodule, those changes will quite possibly be lost the next time you run git submodule update."* The superproject stores a specific commit SHA (mode `160000`), not a branch, so nothing keeps the new commit reachable.

- **Mitigation:** the mechanism NEVER commits into a detached HEAD. Before any commit in a submodule under rule (a), it MUST first ensure the submodule is ON its tracking branch (`git symbolic-ref -q HEAD` resolves to `refs/heads/<branch>`); if detached, it checks out the tracking branch first (fast-forwarding the branch to the current detached SHA only if that is a strict ancestor, else it is a conflict → stop, no writes). The per-submodule state machine (Part B2 §4) makes `detached` an explicit, non-committable state.

R – Auto-updating a deliberately-PINNED non-main branch (the rule (b) violation)

- **Failure mode:** an operator has checked a submodule out to `feature/mistiq-vader` to do work; the auto-sync "helpfully" fetches+resets it to `main` , destroying the operator's in-progress branch state / uncommitted work.
- **Git-level cause:** `git submodule update --remote` with `submodule.<name>.update` other than `none` will move HEAD; a naïve "always pull main" loop ignores which branch is checked out ([S3] update modes).
- **Mitigation:** the per-submodule state machine has a `pinned` state. A submodule whose current branch ≠ its configured tracking branch (`submodule.<name>.branch` , default `main`) AND whose checkout was operator-initiated is `pinned` : rule (a) auto-update is SKIPPED entirely; only rule (c) parent-pointer bookkeeping and rule-(a)-cadence-merge-INTO-the-pinned branch (§11.4.188, background, ff-only, conflict-stops) apply, never a reset onto main. Pin detection is recorded explicitly in `.docs_chain/submodule_pins.json` (see §4.3) so "deliberate" is a fact, never a guess (§11.4.6).

R – Force-push data loss (STRICTLY FORBIDDEN)

- **Failure mode:** to "resolve" a divergence between a submodule's local branch and its remote (or between two mirrors), a naïve implementation force-pushes and silently deletes commits another track/session pushed.
- **Git-level cause:** `git push --force` / `--force-with-lease` / `+<ref>` rewrites the remote ref, discarding commits not in the local history.

- **Mitigation:** §11.4.113 ABSOLUTE — force-push is STRICTLY FORBIDDEN with NO exception (no operator-approval path). The mechanism uses the §11.4.113 6-step integration procedure: fetch all remotes → set base to latest `main / master` tip → MERGE local changes onto it (union, no `-s ours`, no reset, no rebase of a shared branch) → resolve conflicts with zero markers → commit the merge → ff-only push to ALL upstreams. Because the merge commit descends from every mirror tip, every push is a fast-forward and force is never necessary. A §11.4.109-class PreToolUse guard blocks `push --force / --force-with-lease / push +<ref>` at the tool-call boundary.

R — Branch divergence between mirrors (multi-upstream § .)

- **Failure mode:** owned repos have multiple upstreams (github, origin, upstream, vasicdigital); mirror A advanced by track-1, mirror B by track-2; a push to B rejects (non-fast-forward) and the sync stalls or (worse) forces.
- **Git-level cause:** §2.1 multi-upstream push is the norm; independent pushes to different mirrors create divergent tips.
- **Mitigation:** before every push, `git fetch --all --prune --tags` (§11.4.37/§11.4.71 — the most-advanced mirror tip is the base); merge every non-empty `HEAD..<mirror>/<branch>` range locally (R3 procedure); then a single ff-only fan-out push to every upstream lands on all. If a mirror advanced again mid-run, re-fetch that mirror, re-merge, re-validate, re-push (bounded retry, then park per §11.4.101). Captured per-mirror push evidence goes in `git_sync_evidence.json`.

R — The on-`main` auto-update rule vs the PINNED-branch rule interaction

- **Failure mode:** ambiguity — a submodule is on `main` in track-1 but `pinned` to a feature branch in track-2 (same URL, different checkouts). A single global "this submodule is on-main" verdict is wrong for one of the two copies.
- **Git-level cause:** the branch-checkout state is per-worktree/per-copy (a property of each physical checkout), not a property of the submodule URL.

- **Mitigation:** the branch-tracking-vs-pinned state machine is **per physical copy**, keyed by `(copy_path)`, NOT by URL. The fan-out index (§5) is keyed by URL for *content* convergence (which SHA), but the *update policy* (auto-update vs pin) is decided per copy from that copy's own HEAD state. A URL can therefore have some copies auto-updating on main and others pinned; each is handled by its own state.

R — The clean-`git status` invariant vs unrelated local edits

- **Failure mode:** the auto-sync runs `git add -A && commit` in a submodule that also holds an operator's unrelated uncommitted edit, sweeping the edit into the sync commit (the §11.4.84 mutation-residue class, and a §11.4.30 `git add -A` footgun).
- **Git-level cause:** `git add -A` stages everything; an in-flight edit or a paired-mutation marker (`// always pass` , `MUTATED for paired`) gets committed.
- **Mitigation:** §11.4.84 working-tree quiescence is a PRECONDITION for any sync in a copy. The mechanism (i) `git status --porcelain` the submodule; (ii) if there is ANY unaccounted modified/untracked entry beyond the mechanism's own declared scope → ABORT that copy (exit non-zero, no writes), surface it (§11.4.66); (iii) it NEVER uses `git add -A` — it stages only the specific gitlink / declared paths; (iv) it greps for mutation markers before any stage (§11.4.84). A dirty submodule is a `dirty` state in the state machine — non-committable.

R — Fan-out inconsistency across duplicate copies of the same submodule

- **Failure mode:** submodule X (URL U) exists at `/mnt/track1/atmosphere/tools/helixqa/...` AND `/mnt/track2/atmosphere/tools/helixqa/...` (and possibly nested inside another submodule). A change lands+pushes on one copy; the other copies keep stale gitlinks → builds diverge, the operator's "all copies must auto-update together" rule is violated.
- **Git-level cause:** each superproject records its OWN gitlink SHA independently; git has no built-in "propagate to sibling checkouts of the same URL" operation.

- **Mitigation:** a **URL-keyed fan-out index** (§5). After a child change is pushed to all mirrors, the mechanism enumerates every OTHER copy of URL U in the registry, `git fetch` es each, updates it (per that copy's own state — auto-update if branch-tracking, or record a parent-pointer bump if pinned), and records the new gitlink in each hosting superproject. Convergence is verified by a fan-out invariant: after a run, every branch-tracking copy of U resolves HEAD to the same pushed SHA (captured in evidence). Copies that are `pinned` are reported (not force-converged) so the operator sees the divergence is intentional.

R — Concurrent writers / races between parallel tracks

- **Failure mode:** track-1 and track-2 both run the sync simultaneously against overlapping submodule URLs; interleaved fetch/merge/push corrupt each other's staged state or produce a lost update.
- **Git-level cause:** §11.4.176/§11.4.178 — parallel tracks; a shared object store (if worktrees) or independent clones both pushing to the same mirror.
- **Mitigation:** an advisory **exactly-once claim registry keyed by submodule URL** (§11.4.176 A) — a track claims URL U for the duration of its sync via a lockfile (`.docs_chain/locks/submodule/<url-hash>.lock` , holder PID recorded); another track targeting U serialises (non-blocking try-lock; if held-and-live, skip+requeue). Per §11.4.179, tracks SHOULD be own- `.git` isolated clones so a stale lock in one cannot freeze the others. Cross- track push conflicts collapse to the R4 fetch-merge-ff-push path. See also R16 (lock death).

R — Worktree shared-object-store hazard (multi-checkout of a superproject)

- **Failure mode:** if the multi-track layout uses `git worktree` (one common `.git`) rather than independent clones, a submodule operation in one worktree corrupts or confuses the submodule state of another, and a stale shared lock freezes all worktrees at once.

- **Git-level cause:** [S2] — *"Multiple checkout in general is still experimental, and the support for submodules is incomplete. It is NOT recommended to make multiple checkouts of a superproject."* All `refs/heads/*` and the object store are shared via `$GIT_COMMON_DIR` ; a branch can be checked out in only one worktree; `HEAD / index` are per-worktree but refs are shared.
- **Mitigation:** the design **REQUIRES** the §11.4.179 own- `.git` isolated-clone (CoW/reflink) layout for parallel tracks and **REFUSES** to operate on a submodule reached through a shared- `common-dir` worktree superproject (detected via `git rev-parse --git-common-dir` resolving outside the copy's own tree → hard error, "run under §11.4.179 isolated clones"). This turns a git-documented "not recommended / incomplete" hazard into an explicit refusal, not a silent risk. **UNCONFIRMED:** whether the current `/mnt/trackN` layout is ¹⁰worktree-shared or isolated- clone on THIS host — the mechanism detects it at runtime rather than assuming (§11.4.6).

R — Partial / network-interrupted fetch or push

- **Failure mode:** a fetch/push is interrupted (network drop, SIGKILL) mid-transfer, leaving the submodule with a partially-updated ref/objects or a mirror pushed while a sibling mirror is not (breaking §2.1 all-mirrors consistency).
- **Git-level cause:** `git fetch / git push` are not transactional across multiple refspecs/ remotes; a crash between mirror-A-push and mirror-B-push leaves them divergent.
- **Mitigation:** (i) fetch is idempotent — a re-run simply re-fetches; the mechanism never commits on the basis of a partial fetch (it re-verifies the tip after fetch). (ii) Push fan-out records per-mirror success in `git_sync_evidence.json` ; a crash leaves some mirrors behind, and the NEXT run's R4 fetch-merge-ff-push brings the laggards forward (ff-only, no loss). (iii) The parent-pointer commit is deferred until the child push to ALL mirrors is confirmed (R11 ordering) — a partial child push means the parent pointer is NOT yet advanced, so no dangling gitlink is published. Composes with §11.4.88 detached background push + per-mirror flock.

R — Parent-pointer commit ordering (child-before-parent)

- **Failure mode:** the superproject records+pushes a new gitlink SHA before the submodule commit is pushed to the submodule's own remote → a clone of the superproject cannot resolve the gitlink (`fatal: reference is not a tree`), because the referenced submodule commit does not exist on any submodule mirror.
- **Git-level cause:** [S1] — *"you have to push the submodule changes before the superproject changes."* `git push --recurse-submodules=check` fails if a submodule contains changes not on any remote; `=on-demand` pushes submodules first.
- **Mitigation:** the fan-out algorithm has a strict topological ordering: for each affected hosting superproject, the child submodule commit MUST be pushed to ALL its mirrors and confirmed (evidence recorded) BEFORE the parent stages/commits/pushes the new gitlink. The mechanism additionally runs `git push --recurse-submodules=check` (or the equivalent reachability probe) as a belt-and-suspenders gate — a superproject push that would leave a ^{1 2} gitlink unresolvable is refused. Recursion applies at every nesting level (grandchild before child before parent).

R — Hook re-entrancy / infinite loop (sync→commit→hook→sync...)

- **Failure mode:** the sync is triggered by a git hook (post-commit / post-merge / a watch daemon on `.git`); its own commit fires the hook again, which re-triggers the sync — an infinite loop that pins a CPU and can spew commits.
- **Git-level cause:** git hooks fire on the very events the sync produces; an fsnotify watch on the repo sees the sync's own writes.
- **Mitigation:** a run-scoped **re-entrancy guard**: the mechanism writes `.docs_chain/SYNC_IN_PROGRESS` (holder PID + start epoch) at run start and removes it at run end (in a `trap ... EXIT`); any hook/watch trigger that sees a live guard is a NO-OP. Docs Chain early-cutoff (§4 of ARCHITECTURE) is the second layer: after the sync's own commit, the node hash equals the just-written value, so the next pass sees "not dirty" and stops — no loop (the exact

mechanism that already prevents `md↔db` oscillation). A per-run visited-set +
¹³ max-iteration guard (ARCHITECTURE §6.1) is the third layer: fail loudly, never spin.

R — Nested-submodule recursion depth + `.gitmodules` graph cycles

- **Failure mode:** the constitution submodule has its own `.gitmodules`; a naïve recursive walk either recurses unboundedly, or a (mis-configured) submodule graph containing a cycle (`A` includes `B` includes `A`) causes infinite recursion.
- **Git-level cause:** [S3] — `git submodule update --init --recursive` and `foreach --recursive` walk the `.gitmodules` graph; nothing intrinsically forbids a cycle in that graph, and §11.4.28(C) forbids nested own-org chains precisely because they are hazardous.
- **Mitigation:** (i) the recursive walk maintains a **visited-URL set**; re-encountering a URL aborts that branch with a "cycle in `.gitmodules` graph" error (exit config-error, like Docs Chain's Kahn cycle refusal, exit 4). (ii) A **max-depth guard** (configurable, default e.g. 8) bounds recursion. (iii) The design HONORS §11.4.28(C): owned-submodule nested chains are forbidden in general; the ONLY sanctioned nesting is the depth-1 constitution-submodule carve-out (`constitution/submodules/*`) — the walker treats depth-1 constitution
¹⁴ nesting as expected and anything deeper in an owned repo as a §11.4.28(C) violation to REPORT, not silently traverse.

R — Submodule URL drift / renamed paths

- **Failure mode:** a submodule's URL in `.gitmodules` changed (org rename, SSH → HTTPS) but the local `.git/config` still points at the old URL; fetch/push hit the wrong (or dead) remote, or the fan-out index keys two physically-same submodules under two different URL strings and fails to converge them.
- **Git-level cause:** [S3] — `.gitmodules` (tracked) and `.git/config` (local) hold the URL independently; `git submodule sync [--recursive]` reconciles `.gitmodules` → `.git/config` but only for already-initialised submodules.

- **Mitigation:** the mechanism runs `git submodule sync --recursive` before any fetch so `.git/config` reflects `.gitmodules`. The fan-out index keys copies by a **canonicalised remote URL** (normalise `git@github.com:org/Repo.git` and `https://github.com/org/Repo.git` and trailing `.git` to one canonical form) so the same logical repo under two URL spellings converges as one. A renamed path is detected (gitlink path no longer matches¹⁵ `.gitmodules`) and REPORTED (operator decision, §11.4.66), never auto-guessed.

R — Uninitialised / absent submodules

- **Failure mode:** a submodule is declared in `.gitmodules` but not initialised in a given copy (`git submodule status prefix -`); the sync tries to fetch/commit inside an empty directory and errors or, worse, treats "no gitlink" as "delete the submodule".
- **Git-level cause:** [S3] — `git submodule status prefix -` = not initialised; the working tree has no `.git` for it.
- **Mitigation:** the mechanism classifies each declared submodule via `git submodule status` prefixes: `-` (uninitialised) → SKIP with reason (`hardware_not_present` -analogue, §11.4.69 skip vocabulary), never a delete; `+` (checked-out SHA ≠ index SHA) → the `dirty / drift` state; `U` (merge conflict)¹⁶ → the `conflict` state (stop, no writes, operator resolves); `(clean)` → normal. No auto-init unless the copy's context declares `auto_init: true`.

R — The git-lock family on holder death (index.lock / HEAD.lock / refs/ / .lock)

- **Failure mode:** a sync process (or a concurrent `commit_all.sh / push_all.sh`) dies holding `.git/index.lock / .git/HEAD.lock / .git/refs/**/*.lock`; every subsequent git op in that repo (and, under a shared worktree common-dir, every worktree, R9) freezes with *"Another git process seems to be running"* — the exact 9-hour all-track freeze forensic anchor of §11.4.180.
- **Git-level cause:** git takes an existence-based lock file for index/ref updates and removes it on completion; a crashed holder leaves the file behind (git does NOT auto-release on death).

- **Mitigation:** §11.4.180 stale-lock auto-reap — BEFORE acquiring, the mechanism reaps a PROVABLY-stale lock: recorded-holder PID DEAD (`kill -0 <pid>` fails), OR (no PID) age > threshold AND no live `git/commit_all.sh / push_all.sh` holding that repo. It reaps `.docs_chain/*.lock` plus the git-internal `index.lock / HEAD.lock / refs/**/*.*.lock`. It NEVER removes a live-held lock (that corrupts a concurrent writer — §9.2). Each REAPED/KEPT decision

¹⁷ ◦ reason is logged (`qa-results/.../lock_reap/<ts>.log`) as captured evidence (§11.4.6 — liveness PROVEN via `kill -0`).

R — Destructive-op data-safety (branch reset, gitlink rewrite, submodule de-init)

- **Failure mode:** a merge-onto-main or a fan-out reset loses commits; a submodule de-init drops local work; a bulk gitlink rewrite corrupts the tree.
- **Git-level cause:** any history rewrite / reset / de-init / bulk removal is destructive.
- **Mitigation:** §9.2 zero-risk data safety — BEFORE any destructive op the mechanism takes a hardlinked `.git` mirror backup (`cp -al .git <backup>/repo.git.mirror` , zero disk, <2 s), records metadata (refs/tags/submodule-pointers/HEAD/tree-hash), defines the expected post-op state, runs the op (never `--no-verify` , never auto-force), runs the post-op gate (HEAD tree identical where content should be preserved; all tags/pointers intact), restores immediately on any mismatch, and writes an audit trail. Merges are the ONLY sanctioned way to reconcile divergence (no reset/rebase of shared branches).

R — Constitution-submodule change workflow specificity (§ . .)

- **Failure mode:** the mechanism treats the constitution submodule like any other and skips the §11.4.26 pipeline (fetch+pull FIRST, careful conflict resolution, cascade, bump pointer, post- merge full validation) — landing a constitution change on one upstream but not others (a §2.1 + §11.4.26 violation) or leaving the consumer pointer stale.
- **Git-level cause:** the constitution submodule is a governance source-of-truth with a mandatory update pipeline distinct from ordinary code submodules.

- **Mitigation:** the constitution submodule is a FIRST-CLASS special case in the config (`constitution_workflow: true` on its node). For it the mechanism runs the full §11.4.26 ordered pipeline: `fetch+pull- --ff-only FIRST` → `apply` → `validate` (no conflict markers, cross-reference consistency) → `commit+push` to ALL upstreams → `post-merge git submodule update --remote --init + cascade verifier` → `bump the consumer .gitmodules pointer in the SAME`
¹⁹ `parent commit. Never git add -A inside the constitution submodule`
^{11 4 188} (§11.4.30).

R — main→feature merge cadence drift (§ . .)

- **Failure mode:** a long-lived pinned feature-branch submodule (rule (b)) drifts far from `main` ; the eventual back-merge is a giant conflict storm and integration risk.
- **Git-level cause:** a feature branch that never integrates trunk diverges monotonically.
- **Mitigation:** §11.4.188 — for every live pinned feature-branch copy, the mechanism regularly (after every trunk tag / $\geq$ daily / before a significant new chunk) `git merge origin/main` INTO the pinned branch (NEVER rebase a shared branch; `fetch-first`; §9.2 backup before a large/ risky merge; conflicts stop with zero markers; `post-merge smoke GREEN + conflict-marker scan`
²⁰ `empty + no-lost-commit, captured`). This is a MERGE-INTO-the-pinned-branch (keeping the pin), distinct from the forbidden rule-(b) `reset-onto-main` (R2).

R — Diamond / shared dependency (same submodule reached by two parents)

- **Failure mode:** URL U is a submodule of both parent P1 and parent P2 (a diamond); a fan-out update advances U and must record the new gitlink in BOTH P1 and P2, but a partial run updates only P1 → the two parents disagree on U's version.
- **Git-level cause:** the submodule graph is a DAG, not a tree; one child can have many parents.

- **Mitigation:** the fan-out index (§5) enumerates ALL hosting superprojects of U (not just one) and the parent-pointer-bump is applied to EVERY hosting parent in one atomic run (Docs Chain all-or-nothing commit, §8): either every parent records the new gitlink or none does (rollback, exit 3). The topological order^{2 1} (R11) is respected across the whole diamond (U pushed before P1 and P2 both bump).

R — `.gitmodules` -vs- `.git/config` -vs- index three-way inconsistency

- **Failure mode:** the tracked `.gitmodules`, the local `.git/config`, and the index gitlink disagree (path present in `.gitmodules` but no gitlink in index, or vice-versa); the mechanism acts on a phantom submodule.
- **Git-level cause:** these three stores are updated by different commands and can drift.
- **Mitigation:** at run start the mechanism cross-checks all three (a §11.4.110-class clash detector) and REFUSES to sync a submodule whose three views disagree (REPORT, operator decides §11.4.66) — it does not guess which is authoritative (§11.4.6).

R — Evidence/analyzer bluff (the sync reports success but did nothing / did wrong)

- **Failure mode:** the sync exits 0 and reports "all in sync" while a copy actually still holds a stale gitlink, or a push silently landed on only one mirror — a §11.4 PASS-bluff at the sync layer.
- **Git-level cause:** an exit code alone does not prove the sink-side state.
- **Mitigation:** §11.4.69 sink-side evidence — every sync PASS cites a captured `git_sync_evidence.json` (per-copy pre/post gitlink SHA, per-mirror push confirmation via `git ls-remote`, fan-out convergence proof, lock-reap log). `verify` (read-only) is the deterministic gate (§11.4.50). The analyzer that reads the evidence is SELF-VALIDATED (§11.4.107(10)): a golden-good fixture (all copies converged, all mirrors pushed) MUST PASS; a golden-bad fixture (one copy stale / one mirror unpushed / a dangling gitlink) MUST FAIL and pinpoint the offender. An analyzer that passes its golden-bad is itself a bluff and a release blocker.

PART B – Design: reactive submodule auto-sync as a Docs Chain extension

. Why Docs Chain, not a new script

Docs Chain (§11.4.106) is the canonical mechanical sync engine; §11.4.106(A) forbids ad-hoc `sync_*` scripts and mandates using the engine. The submodule `sync` is *structurally the same problem* Docs Chain already solves for documents — a dependency graph where a change to one member must propagate, deterministically and atomically, to every dependent member — so it maps onto Docs Chain's primitives with a new node kind and transform family rather than a bespoke loop. Reusing the engine gives us, for free: content-hash change detection (§11.4.86), Kahn cycle rejection, declared-authority bidirectional `sync` with the both-dirty⇒conflict rule (§11.4.6), atomic stage+rename+rollback (§8 / §9.2), `verify` as the deterministic pre-build gate (§11.4.50), and captured evidence (§11.4.69). The engine stays project-agnostic (§11.4.28); the ATMOSphere submodule roster is consumer data in `.docs_chain/contexts/*.yaml`.

. New node kinds and edge/transform family

Extend the Docs Chain node-kind enum (CONFIG_SCHEMA §3.1) with:

New node kind	Role	Direction	"content hash" is...
<code>git_submodule</code>	one physical checkout of a submodule at a path	input + derived	canonical of (remote-URL, current-branch, HEAD-SHA, tracking-branch, dirty?, init-state)
<code>git_super-project</code>	a hosting repo (main repo, or a parent submodule)	input + derived	canonical of (recorded gitlink SHAs of its children + its own branch/HEAD)
<code>git_remote_ref</code>	a mirror's branch tip (sink-side, read via <code>git ls-remote</code>)	sink	the remote SHA reported by <code>ls-remote</code>

Extend the transform builtins (CONFIG_SCHEMA §5.1) with a **git-adapter family** (all Go builtins in a new `internal/gitsync` package, so the engine stays self-contained and testable):

New builtin	Maps	Effect
<code>git-fetch-all</code>	<code>git_submodule</code> → <code>git_submodule</code>	<code>git submodule sync --recursive</code> + <code>git fetch --all --prune --tags</code> ; re-compute node hash
<code>git-merge-onto-main</code>	<code>git_remote_ref</code> → <code>git_submodule</code>	§11.4.113 6-step: base=latest main tip, merge local, ff-only (never force)
<code>git-commit-branch</code>	<code>git_submodule</code> → <code>git_submodule</code>	ensure on tracking branch (R1), stage ONLY declared paths (never <code>-A</code> , R6), commit
<code>git-push-all-mirrors</code>	<code>git_submodule</code> → <code>git_remote_ref</code>	ff-only fan-out push to all upstreams (§2.1); per-mirror <code>ls-remote</code> confirm
<code>git-record-gitlink</code>	<code>git_submodule</code> → <code>git_superproject</code>	stage+commit the parent's new gitlink SHA (child-before-parent, R11)
<code>git-fanout-converge</code>	<code>git_submodule</code> → <code>git_submodule[]</code>	fetch+update every OTHER copy of the same canonical URL (R7/§5)
<code>git-constitution-flow</code>	<code>git_submodule</code> → <code>git_superproject</code>	the §11.4.26 ordered pipeline for the constitution node (R18)

Every git-adaptor transform obeys the Docs Chain `exec /builtin` contract (CONFIG_SCHEMA §5.2): read only declared inputs, write only staged temp state, exit non-zero → rollback (exit 3), byte-stable/deterministic (§11.4.50). Because git ops touch the real repo (not a temp file), "staging" for these transforms means: do the mutation on a §9.2 hardlinked-backed working state and only *publish* (push / advance parent pointer) in the commit phase; any pre-commit error restores from the backup (R17).

. Event model – what triggers a sync

A sync run is triggered by any of (all funnel into the same `git_sync <context>` entry point):

1. **Explicit** — `docs_chain git-sync <context> / --all` (the operator or the §11.4.187 ruler invokes it; the primary path).
2. **Cadence** — a scheduled tick (§11.4.188 daily / after every trunk tag) for the merge-INTO- feature cadence and the rule-(a) main pull.
3. **Post-commit signal** — after ANY submodule commit+push lands anywhere (rule (c)), the committing process appends a `child-changed <url> <sha>` event to the Docs Chain append-only event stream (§11.4.116); the sync consumes it and fans out. This is a SIGNAL, not a git hook that re-enters (R12) — the re-entrancy guard (§6) makes a self-triggered signal a no-op.
4. **Watch (optional, later phase)** — an fsnotify daemon (`internal/watch` , ARCHITECTURE §11) on the tracked `.gitmodules` / gitlink set, debounced, runs `git-sync` on settle; guarded by §6 against self-triggering.

Every trigger is idempotent: a run with nothing dirty exits 0 with no writes (early-cutoff, §4).

. Per-copy submodule state machine (branch-tracking vs pinned)

The policy for each physical copy is decided from that copy's own HEAD state (R5 — per copy, not per URL). States and transitions:

```

git submodule status prefix / HEAD probe
|
+-----+-----+-----+-----+
+
|          |          |          |
| ' ' clean '+' drift 'U' conflict '-' uninit
detached HEAD
|          |          |          |
|          v          v          v          v
v
[CLEAN]      [DRIFT]      [CONFLICT]      [UNINIT]
[DETACHED]
|          |          |          |
|
branch == tracking? |          STOP, no writes      SKIP w/ reason
checkout tracking
| |          |          operator resolves (never delete)
branch (ff-only if
yes no          |          (§11.4.66)      (§11.4.69 skip)
ancestor) else
| |          v
-> [CONFLICT]
| |          record parent-pointer
| |          drift; do NOT reset
| v
| [PINNED] (operator on non-main branch, rule (b))
| |
| +-- rule (a) auto-update: SKIPPED
| +-- rule (c) parent-pointer bookkeeping: applies
| +-- §11.4.188 merge origin/main INTO pinned branch (bg, ff,
conflict-stops)
|
v
[BRANCH-TRACKING] (on the tracking branch, rule (a))
|
+-- git-fetch-all -> git-merge-onto-main (ff-only, §11.4.113)
+-- if local commits: git-commit-branch -> git-push-all-mirrors
(§2.1)
+-- git-fanout-converge (rule (c), §5) -> git-record-gitlink (child-
before-parent, R11)

```

. State determination is a FACT, not a guess

branch-tracking vs pinned is determined mechanically:

- current branch := `git -C <copy> symbolic-ref -q --short HEAD` (empty ⇒ `DETACHED`);
- tracking branch := `submodule.<name>.branch` from the hosting `.gitmodules` (default `main`);
- if `current == tracking` ⇒ `BRANCH-TRACKING`;
- if `current != tracking` AND the copy is recorded in `.docs_chain/submodule_pins.json` as an operator-initiated pin ⇒ `PINNED`;
- if `current != tracking` and NOT recorded as a pin ⇒ **ambiguous** → STOP + REPORT (§11.4.66), never guess (§11.4.6). (An operator registers a deliberate pin explicitly, e.g.
`docs_chain git-pin <copy> <branch> --reason "..."`, writing
`submodule_pins.json`. This makes "deliberate" auditable — the R2 mitigation.)

. `submodule_pins.json` (per copy, tracked)

```
{
  "pins": [
    { "copy_path": "device/rockchip/atmosphere/mpv-player",
      "branch": "feature/mistiq-vader",
      "reason": "T2 ATM-xxx work",
      "by": "User", "on": "2026-07-14",
      "canonical_url": "github.com/milos85vasic/ATMOSphere-MPV-Player" }
  ]
}
```

. Fan-out propagation algorithm (keyed by canonical submodule URL)

The registry `.docs_chain/submodule_registry.json` maps each canonical URL to every physical copy and every hosting superproject:

```

{
  "github.com/HelixDevelopment/HelixQA": {
    "copies": [
      { "copy_path": "tools/helixqa/HelixQA", "host_superproject": "<root>" },
      { "copy_path": ".../track2/.../tools/helixqa/HelixQA", "host_superproject": "<root2>" }
    ]
  }
}

```

Algorithm `fanout(url U, pushed_sha S)` :

```

1. reap stale locks for U (§11.4.180 / R16); try-claim advisory lock on U
   (§11.4.176 / R8).
   held-and-live -> requeue U, return.
2. for each copy C of U in registry:
   a. §11.4.84 quiescence check on C; not-quiescent -> ABORT C, report
   (R6).
   b. git submodule sync --recursive ; git fetch --all (R14).
   c. classify C's state (§4). CONFLICT/UNINIT/ambiguous -> skip C w/
   reason, report.
   d. BRANCH-TRACKING -> fast-forward C's tracking branch to S (ff-only;
   if not ff, R4
       merge-onto-latest then re-derive; never force, R3).
       PINNED -> do NOT move C's branch; only record the parent-pointer
       bump (below).
   e. stage the new gitlink S in C's host_superproject (child-before-
   parent already satisfied:
       S was pushed to all U-mirrors before fanout was entered, R11).
3. topologically order all touched host_superprojects (children before
   parents, R11/R20);
   for each, in order: git-record-gitlink (commit the new child SHA), then
   recurse UPWARD
   (the host superproject is itself a git_submodule of ITS parent ->
   fanout its own gitlink
       change up the chain, bounded by the visited-set + max-depth, R13).
4. atomic commit phase (Docs Chain §8): all staged gitlink commits +
   pushes land together or
   none (rollback, exit 3). fan-out convergence invariant asserted: every
   BRANCH-TRACKING copy
   of U now resolves HEAD == S; every PINNED copy reported as
   intentionally divergent.
5. release U's advisory lock; write git_sync_evidence.json.

```

Canonical-URL normalisation (R14): strip scheme (`git@...:` vs `https://.../`), lowercase host, strip trailing `.git` , so all spellings of one repo key to one entry.

. Re-entrancy guard (the sync-triggers-commit-triggers-sync loop)

Three defence layers (R12):

1. **Run-scoped flag** — `.docs_chain/SYNC_IN_PROGRESS` (holder PID + start epoch) written at run start, removed in `trap ... EXIT` . Any event/hook/watch trigger observing a LIVE guard (holder PID alive via `kill -0` , §11.4.180 liveness) is a NO-OP and returns immediately. A STALE guard (dead holder) is reaped like any other lock.
2. **Early-cutoff** (ARCHITECTURE §4) — after the sync's own commit, the `git_submodule` / `git_superproject` node hash equals the just-written value; the next pass computes "not dirty" and stops. This is the same mechanism that prevents `md↔db` oscillation.
3. **Per-run visited-set + max-iteration guard** (ARCHITECTURE §6.1) — belt-and-suspenders; fail loudly on any unexpected oscillation, never spin.

. Change-detection (content-hash, not mtime — § . .)

A `git_submodule` node is dirty when its freshly-computed hash differs from the hash stored in `state.json` . The hash is `sha256` of a canonical tuple, NOT the working-tree mtime:

```

node_hash(git_submodule) = sha256( canonical_url ||
                                   current_branch ||
                                   HEAD_sha      ||
                                   tracking_branch||
                                   dirty_bool    ||
                                   init_state  )

node_hash(git_superproject) = sha256( sorted list of (child_path,
recorded_gitlink_sha) ||
                                   own_branch || own_HEAD_sha )

node_hash(git_remote_ref) = the SHA from `git ls-remote <mirror>
<branch>`

```

This directly encodes §11.4.86: `git checkout` resets file mtime, so an mtime fingerprint silently misses real drift; the gitlink-SHA + branch-state hash is the drift-proof signal (the same reason the roster/corpus fingerprint hashes the sorted member list, not mtimes).

. Composition with the Docs Chain context-YAML model

A submodule-sync context is an ordinary `.docs_chain/contexts/<name>.yaml` using the new node kinds and git-adapter builtins. It composes with the existing exit-code + atomicity + verify model (ARCHITECTURE §12). Illustrative (PLANNED contract — not parsed today):

```

context: submodule_sync_owned      # consumer-owned data (§11.4.28); engine is generic
description: Recursive reactive auto-sync of owned submodules (rules a/b/c)
nodes:
  # each owned submodule copy is a git_submodule node
  mpv_copy:    { kind: git_submodule, path: device/rockchip/atmosphere/mpv-player }
  presenter_copy: { kind: git_submodule, path: device/rockchip/atmosphere/presenter }
  helixqa_copy: { kind: git_submodule, path: tools/helixqa/HelixQA }
  constitution: { kind: git_submodule, path: constitution, constitution_workflow: true }
  root:        { kind: git_superproject, path: . }
  # sink-side mirror tips (read-only, §11.4.69)
  mpv_github:  { kind: git_remote_ref, path: "mpv-player#github/main" }
edges:
  # rule (a): on-main submodule -> fetch, merge-onto-main (ff-only), push all mirrors
  - { type: derive-from, from: mpv_github, to: mpv_copy, transform: git-merge-onto-main }
  - { type: derive-from, from: mpv_copy, to: mpv_github, transform: git-push-all-mirrors }
  # rule (c): a pushed child change fans out + records the parent gitlink (child-before-parent)
  - { type: derive-from, from: mpv_copy, to: root, transform: git-record-gitlink }
  # constitution submodule uses the §11.4.26 ordered pipeline
  - { type: derive-from, from: constitution, to: root, transform: git-constitution-flow }
transforms:
  git-merge-onto-main: { builtin: git-merge-onto-main }
  git-push-all-mirrors: { builtin: git-push-all-mirrors }
  git-record-gitlink: { builtin: git-record-gitlink }
  git-constitution-flow: { builtin: git-constitution-flow }
policy:
  # NEW optional block for git-sync contexts
  force_push: forbidden      # §11.4.113 — enforced, no override
  quiescence_required: true  # §11.4.84
  stale_lock_reap: true      # §11.4.180
  backup_before_destructive: true  # §9.2
  max_recursion_depth: 8     # R13
  pins_file: .docs_chain/submodule_pins.json # rule (b)
  registry_file: .docs_chain/submodule_registry.json # §5 fan-out

```

The `sync` -edge both-dirty conflict rule (ARCHITECTURE §5) carries over verbatim: if a copy AND its mirror are both dirty in one run (a concurrent edit both sides), the run STOPS with a conflict (exit 2, no writes), operator resolves (§11.4.66) — no silent merge (§11.4.6).

. Ordering guarantee (child-before-parent-pointer) – restated as an invariant

For every affected superproject P and its child submodule C:

INV-ORDER: P's new gitlink SHA for C is committed+pushed ONLY AFTER C's commit is confirmed present on ALL of C's mirrors (verified via `git ls-remote`). Recursively: a grandchild is confirmed on its mirrors before the child records it, before the child is pushed, before the parent records the child.

Enforced by the topological order in `fanout` (§5 step 3) PLUS a belt-and-suspenders `git push --recurse-submodules=check` reachability probe (R11) before any superproject push: a push that would publish an unresolvable gitlink is refused.

. Anti-bluff surface

- **Captured evidence (§11.4.69):** every `git-sync` run writes `qa-results/docs_chain/git-sync/<run-id>/git_sync_evidence.json` — per-copy pre/post gitlink SHA, per-mirror push confirmation (`ls-remote`), fan-out convergence proof, lock-reap log, backup path (§9.2). A PASS with no evidence file is a §11.4 bluff.
- **verify gate (§11.4.50):** `docs_chain git-verify <context>` is read-only — it reports drift (a stale copy, an unpushed mirror, a dangling gitlink, a pin that drifted past its §11.4.188 cadence) and writes NOTHING; it is the pre-build/pre-merge gate.
- **Self-validated analyzer (§11.4.107(10)):** the evidence analyzer ships a golden-good fixture (all copies converged, all mirrors pushed, no dangling gitlink → MUST PASS) AND a golden-bad fixture (one copy stale / one mirror unpushed / a dangling gitlink / a force-push attempt → MUST FAIL, pinpointing the

offender) AND a negative-control (a legitimately-PINNED divergent copy → MUST PASS, not mis-flagged as drift). An analyzer that passes its golden-bad, or fails golden-good / the negative-control, is itself a bluff and a release blocker.

- **Deterministic consistency (§11.4.50):** N identical runs on an in-sync tree produce identical exit codes AND identical evidence hashes (no writes on a no-op run).

• Honest boundary (§ . .)

This design proves **internal git-graph consistency + safe convergence under the operator's three rules**. It does NOT prove that a submodule's CODE builds or that a bumped gitlink is functionally correct — that stays the §11.4.108 runtime-signature / §11.4.40 retest job. The mechanism guarantees *no lost commit, no force-push, no dangling gitlink, no stale duplicate, no detached-HEAD loss, no auto-clobber of a deliberate pin, and a clean `git status` on every touched copy* — not that the resulting tree is releasable. UNCONFIRMED items (marked inline): whether the current `/mnt/trackN` layout is worktree-shared (R9) — detected at runtime, never assumed; and the constitution submodule's own nested-submodule push topology — the mechanism reads it live rather than hardcoding it.

3

PART B — Phased implementation plan (tracked follow-on work items)

NONE of the following is implemented this run. This is a PLAN. Each phase is an ordered tracked work item (assign ATM-/CM- IDs per §11.4.54 when queued). Every phase carries: scope, files it would touch, its pre-build gate + paired §1.1 mutation + on-host test (four-layer per §11.4.4(b)), and acceptance criteria. Phases are sequential (a later phase depends on earlier ones); within a phase the sub-work is parallelisable (§11.4.58).

Phase — Spec + RED baseline (no engine code)

- **Scope:** freeze this design; write the failing acceptance spec (the RED test per §11.4.43/§11.4.115) that a not-yet-built `docs_chain git-sync` MUST make GREEN; enumerate the golden-good / golden-bad / negative-control fixtures.
- **Files:** `constitution/docs/research/reactive_submodule_sync/DESIGN.md` (this file), `.../SPEC.md`, `.../fixtures/{golden_good, golden_bad, negative_control}/` (throwaway git trees).
- **Gate:** `CM-SUBMODULE-SYNC-SPEC-PRESENT` (design + spec + fixtures exist).
- **Paired §1.1 mutation:** delete a fixture → gate FAILs.
- **On-host test:** `bash -n / sh -n` clean on any spec scripts (§11.4.67); fixtures are valid git repos (`git -C ... status` succeeds).
- **Acceptance:** design + spec + 3 fixtures committed; RED test exists and FAILS (nothing built yet — the honest RED per §11.4.115).

Phase — `internal/gitsync` core: state machine + content-hash + registry (read-only)

- **Scope:** the pure, side-effect-free core — classify each copy's state (§4), compute `git_submodule / git_superproject / git_remote_ref` node hashes (§7), build the URL-keyed registry (§5) and `submodule_pins.json` reader. NO writes to any repo yet.
- **Files:** `docs_chain/internal/gitsync/{state.go, hash.go, registry.go, pins.go, canonurl.go}`
 - `_test.go` siblings.
- **Gate:** `CM-GITSYNC-STATE-CLASSIFY` (state machine covers clean/drift/conflict/unit/ detached/pinned/ambiguous; canonical-URL normalisation collapses ssh/ https spellings).
- **Paired §1.1 mutation:** flip the `pinned` classifier to auto-update a non-main branch → gate FAILs (R2 guard proven).
- **On-host test:** run against the Phase 0 fixtures; assert classifications + node hashes are deterministic (§11.4.50, N runs identical).

- **Acceptance:** every fixture classifies correctly; a `git checkout` that resets mtime² does NOT change a node hash (§11.4.86 proven); `go test ./internal/gitsync/... -count=3` GREEN.

Phase — Safety primitives: quiescence, lock-reap, § backup, re-entrancy guard

- **Scope:** the guards that make writes safe, still with NO repo-mutating git op — just the lock/backup/guard machinery + their proofs.
- **Files:** `docs_chain/internal/gitsync/{quiescence.go, lockreap.go, backup.go, reentrancy.go}`
 - tests; reuse the constitution `host_session_safety` /lock helpers by reference (§11.4.28), do not copy.
- **Gate:** `CM-GITSYNC-SAFETY-PRIMITIVES` (lock-reap only reaps DEAD-holder locks via `kill -0` ; §9.2 hardlink backup taken before any destructive marker; re-entrancy flag honored).
- **Paired §1.1 mutation:** make lock-reap remove a LIVE-held lock → gate FAILs (§11.4.180 / §9.2 violation caught); strip the re-entrancy flag → the loop-guard test FAILs.
- **On-host test:** kill a fake holder mid-lock (leaving a dead-holder lock) → reap proceeds + logs REAPED + dead PID; a control run with a LIVE holder → KEEPS the lock + waits. Both captured (the §11.4.180 runtime signature).
- **Acceptance:**³ dead-holder reap + live-holder keep both proven with captured logs; §9.2 backup round-trips (restore yields byte-identical `.git`).

Phase — Rule (a): on-main fetch + merge-onto-main (ff-only) + multi-mirror push

- **Scope:** the first repo-mutating transforms — `git-fetch-all` , `git-merge-onto-main` (§11.4.113 6-step, ff-only, NEVER force), `git-commit-branch` (on-branch, never `-A`), `git-push-all-mirrors` (§2.1 fan-out + `ls-remote` confirm). Detached-HEAD → branch guard (R1).

- **Files:** docs_chain/internal/gitsync/transforms_rule_a.go + tests; a §11.4.109-class PreToolUse guard guard-no-force-push referenced (not copied).
- **Gate:** CM-GITSYNC-RULE-A-NO-FORCE (no push --force / --force-with-lease / +ref anywhere; merge-onto-latest-main only; commit never uses git add -A ; detached-HEAD never committed).
- **Paired §1.1 mutation:** inject a git push --force into the push transform → gate FAILs (§11.4.113); change commit to git add -A → gate FAILs (R6).
- **On-host test:** on a throwaway 2-mirror fixture: local commit on main → merge-onto-latest → ff push to BOTH mirrors → ls-remote confirms both tips advanced; a divergent mirror forces a fetch-merge-ff cycle (never force). Captured git_sync_evidence.json .
- **Acceptance:** RED merge-onto-main test flips GREEN; both mirrors converged with NO force; a detached-HEAD commit is refused; deterministic across N runs (§11.4.50).

Phase — Rule (b): PINNED-branch protection + § . . merge-INTO-feature cadence

- **Scope:** the pinned policy — rule (a) auto-update SKIPPED for a pinned copy; the §11.4.188 git merge origin/main INTO the pinned branch (background, ff-safe, conflict-stops, §9.2 backup for large merges, zero markers, post-merge smoke); git-pin / git-unpin CLI writing submodule_pins.json .
- **Files:** docs_chain/internal/gitsync/transforms_rule_b.go , cmd/docs_chain/git_pin.go + tests.
- **Gate:** CM-GITSYNC-PIN-PROTECTION (a pinned copy is NEVER reset onto main; the cadence is a MERGE-INTO-pinned not a rebase; ambiguous non-main-non-pinned STOPS + reports, never guesses).
- **Paired §1.1 mutation:** make a pinned copy auto-update onto main → gate FAILs (R2); replace the cadence merge with a rebase of the shared branch → gate FAILs (§11.4.188 / §11.4.113).
- **On-host test:** a copy on feature/x registered as a pin → a rule-(a) run leaves its branch untouched (evidence: pre==post branch+HEAD); a cadence tick merges origin/main INTO feature/x (ff/merge, no markers, smoke GREEN, no lost commit).

- **Acceptance:** pinned branch survives every rule-(a) run byte-identical; cadence merge lands with zero conflict markers + no lost commit; ambiguous state stops+reports.

Phase – Rule (c): URL-keyed fan-out + child-before-parent + diamond + recursion bound

- **Scope:** the `git-fanout-converge` + `git-record-gitlink` transforms; the §5 fanout algorithm with INV-ORDER (R11), the diamond multi-parent bump (R20), the visited-set + max- depth recursion bound + `.gitmodules` -cycle refusal (R13), and the atomic all-or-nothing commit across all touched superprojects (Docs Chain §8).
- **Files:** `docs_chain/internal/gitsync/{fanout.go, record_gitlink.go, recursion.go}` + tests.
- **Gate:** `CM-GITSYNC-FANOUT-ORDER` (child pushed to ALL mirrors before ANY parent bumps its gitlink; `push --recurse-submodules=check` reachability probe passes; recursion bounded; `.gitmodules` cycle refused).
- **Paired §1.1 mutation:** reorder to bump the parent gitlink before the child push → gate FAILs (R11 dangling gitlink); remove the visited-set → the cycle-fixture recurses → gate FAILs (R13).
- **On-host test:** a 2-copy + diamond fixture: change on copy-1 → fan-out converges copy-2 → BOTH host superprojects record the new gitlink atomically → a fresh clone of each superproject resolves the gitlink (no `reference is not a tree`). Captured convergence proof.
- **Acceptance:** every branch-tracking copy converges to the pushed SHA; pinned copies reported divergent (not force-converged); zero dangling gitlinks (clone-resolves); diamond parents agree.

Phase – Constitution § . . flow + worktree-hazard refusal + three-way clash detector

- **Scope:** `git-constitution-flow` (the §11.4.26 ordered pipeline: fetch+pull-ff FIRST → apply → validate → push all upstreams → cascade → bump consumer pointer in same parent commit); the R9 refusal to operate through a

shared-common-dir worktree superproject; the R21 three-way (`.gitmodules / .git/config /index`) clash detector; the R14 `git submodule sync --recursive` URL-drift reconciliation.

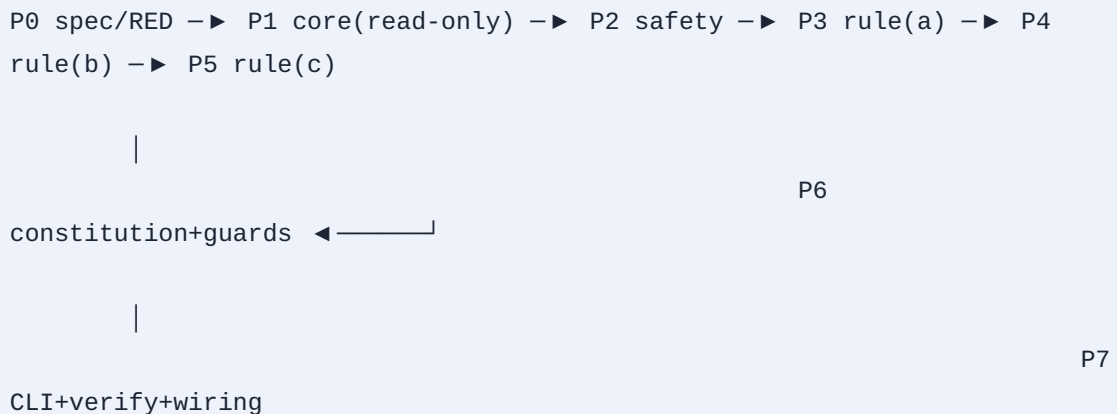
- **Files:** `docs_chain/internal/gitsync/`
`{constitution_flow.go,worktree_guard.go,clash.go}` + tests.
- **Gate:** `CM-GITSYNC-CONSTITUTION-AND-GUARDS` (constitution node runs the §11.4.26 order; shared-common-dir worktree superproject is refused; three-way inconsistency stops+reports).
- **Paired §1.1 mutation:** skip the fetch-first step of the constitution flow → gate FAILs (§11.4.26); allow a shared-common-dir worktree → gate FAILs (R9).
- **On-host test:** a fixture constitution submodule with 4 mirrors → the flow pushes all 4 + bumps the consumer pointer in the same commit; a worktree-shared superproject fixture → refused with the §11.4.179 remediation message.
- **Acceptance:** constitution change lands on ALL upstreams + pointer bumped atomically; worktree hazard refused; three-way clash stops without a guess.

Phase – CLI/daemon + verify gate + self-validated analyzer + ATMOSphere wiring

- **Scope:** `docs_chain git-sync|git-verify|git-graph` CLI (mirroring ARCHITECTURE §12 exit codes 0/2/3/4); the read-only `git-verify` pre-build gate (§11.4.50); the self-validated evidence analyzer (§11.4.107(10), golden-good/golden-bad/negative-control wired into meta- test); the optional `fsnotify` watch daemon (§6-guarded); and the CONSUMER-side ATMOSphere context YAML (`.docs_chain/contexts/submodule_sync_owned.yaml`) + registry + pins seed.
- **Files:** `docs_chain/cmd/docs_chain/git_sync.go` , `docs_chain/internal/gitsync/analyzer.go` , `docs_chain/internal/watch/gitsync_watch.go` ; CONSUMER: `.docs_chain/contexts/*.yaml` , `.docs_chain/submodule_registry.json` , `.docs_chain/submodule_pins.json` , a pre-build gate wiring `git-verify` .
- **Gate:** `CM-GITSYNC-VERIFY-WIRED` + `CM-GITSYNC-ANALYZER-SELFVALIDATED` (verify is read-only and wired into the consumer pre-build; analyzer PASSES golden-good + negative-control, FAILs golden-bad).

- **Paired §1.1 mutation:** make `git-verify` write (not read-only) → gate FAILs; make the analyzer pass its golden-bad fixture → self-validation FAILs (the §11.4.107(10) bluff-proof).
- **On-host test:** end-to-end on the ATMOSphere context against isolated-clone fixtures: a change fans out + records gitlinks + pushes all mirrors + `git-verify` returns 0; then corrupt one copy → `git-verify` returns non-zero pinpointing it. Captured evidence for both.
- **Acceptance:** the Phase 0 RED acceptance spec flips fully GREEN; `git-verify` is a deterministic gate the consumer pre-build invokes; the analyzer is bluff-proof; the ATMOSphere owned-submodule context syncs the three rules end-to-end with captured evidence. Manual-QA final confirmation per §11.4.185 before any release tag.

Phase dependency summary



Each phase composes with §11.4.4(b) four-layer coverage, §11.4.43/§11.4.115 RED-first, §11.4.50 determinism, §11.4.69 captured evidence, §11.4.107(10) self-validated analyzers, §11.4.125/ §11.4.142 code-review-before-build, and §11.4.108 runtime-signature-as-done. The engine changes stay in `docs_chain/` (project-agnostic, §11.4.28); the ATMOSphere submodule roster/registry/ pins are consumer data (§11.4.35).

Sources verified

- [S1] git-scm.com Book — *Git Tools — Submodules* (detached-HEAD default + commit-loss warning; `submodule.<name>.branch` tracking; superproject records exact SHA mode 160000; `git push --recurse-submodules=check|on-demand` child-before-parent; dirty-checkout / merge- conflict warnings): <https://git-scm.com/book/en/v2/Git-Tools-Submodules> — **fetchd + verified this session.**
- [S2] git-scm.com — *git-worktree(1)* (`$GIT_COMMON_DIR` shared object store + shared `refs/heads/*` ; per-worktree `HEAD / index` ; "Multiple checkout ... still experimental ... support for submodules is incomplete ... NOT recommended to make multiple checkouts of a superproject"; `git worktree lock` ; stale `gitdir / repair`): <https://git-scm.com/docs/git-worktree> — **fetchd + verified this session.**
- [S3] git-scm.com — *git-submodule(1)* (`git submodule sync [--recursive]` URL-drift reconciliation; `update --init --recursive` nested recursion; `foreach --recursive` ; `status prefixes - / + / U` ; `submodule.<name>.update = checkout/rebase/merge/none`; `--remote --rebase|--merge`): <https://git-scm.com/docs/git-submodule> — **fetchd + verified this session.**
- Constitution anchors (read in-repo from `constitution/Constitution.md` mirrors this session): §11.4.113 (absolute no-force-push), §9.2 (zero-risk data safety), §11.4.84 (quiescence), §11.4.180 (stale-lock auto-reap), §11.4.26 (constitution-update workflow), §11.4.188 (main → feature merge cadence), §11.4.86 (content-hash fingerprint), §11.4.69 (sink-side evidence), §11.4.107(10) (self-validated analyzer), §2.1 (multi-upstream push), §11.4.28/ §11.4.31 (submodule decoupling / dependency manifest), §11.4.106 (Docs Chain engine), §11.4.176–§11.4.182 (multi-track work-division / track-qualified identity / own- `.git` isolation).
- Docs Chain engine docs (read in-repo this session): `docs_chain/docs/ARCHITECTURE.md` , `docs_chain/docs/CONFIG_SCHEMA.md` (node/edge/ transform model, exit codes, atomic commit, sync-edge authority + both-dirty conflict, early-cutoff, Kahn cycle refusal, CLI/verify).

UNCONFIRMED (per §11.4.6): whether the live `/mnt/trackN` layout is worktree-shared vs isolated-clone on this host (the mechanism DETECTS this at runtime, R9, and refuses the worktree- shared case rather than assuming); the constitution submodule's own nested-submodule push topology (read live, not hardcoded). The `index.lock / HEAD.lock` holder-death behaviour (R16) is grounded in the project's own §11.4.180 forensic anchor (captured evidence) rather than a re-fetched git man-page this session.